

Codex Plan: Legacy 网络融合消融 + 新增 Expr/Pseudotime/SR/Energy/Spatial OT

0. 本计划要解决什么

当前 legacy 网络里, `inter_edges` 表示细胞/空间单元之间的 ligand-receptor 通讯关系。代码中这部分不是单纯 CCI, 而是把 COMMOT 的 CCI 分数和 GRN 中 ligand -> receptor 的调控权重相乘:

$$I_{u_i \rightarrow u_j}^{LR} = s_{CCI}(u_i, u_j, L, R) \times s_{GRN}(L, R)$$

这意味着 legacy 的 inter block 同时使用了 CCI 和 GRN 信息。现在要做两个方向的消融:

1. **Legacy 网络效用消融**: 比较 inter block 里 CCI 与 GRN 的融合方式。
 - baseline: CCI $\times$ GRN, 即当前 `legacy_mixed_grn_cci`。
 - CCI-only: 去掉 inter block 中 GRN 的打分和过滤, 只保留 CCI。
 - additive: 把 CCI $\times$ GRN 改成 CCI + GRN 的加权平均。
2. **PIJ 方法扩展**: 现在 `expression + pseudotime + SR + energy` 效果不错, 进一步加入 `spatial`, 新增:
 - `expr_pseudotime_sr_energy_spatial_ot`

本计划只写开发方案, 不直接修改代码。Codex 需要按本计划修改并跑 smoke test, 确认后再跑全量实验。

1. 当前代码现状确认

1.1 Legacy 网络的关键入口

当前相关文件:

文件	作用
<code>mignet_ce/networks/legacy_mixed_grn_cci.py</code>	legacy network builder, 按 organ / level pair / time point 构造上下层 graph matrix
<code>mignet_ce/graph/builder.py</code>	真正构建 LayerGraph, 包括 <code>intra_edges</code> 和 <code>inter_edges</code>
<code>mignet_ce/features.py</code>	把 LayerGraph 转成 feature matrix, 前半是 intra block, 后半是 inter block
<code>mignet_ce/networks/registry.py</code>	注册 network method
<code>mignet_ce/config.py</code>	NETWORK_METHODS、CLI config、PIJ method 集合
<code>scripts/run_mignet_vertical_ablation.py</code>	network method $\times$ Pij method 消融入口

1.2 当前 inter edge 的问题点

在 `mignet_ce/graph/builder.py` 的 `_build_commot_inter_edges(...)` 中, 当前逻辑大致是:

```
if (ligand_gene, receptor_gene) not in pair_lookup:
    continue
raw_w, norm_w = pair_lookup[(ligand_gene, receptor_gene)]
influence_score = cci_norm * norm_w
```

注意: 这里 GRN 的作用有两层:

1. **过滤作用**: 如果 ligand -> receptor 不在 GRN 里, 直接跳过。
2. **打分作用**: 如果存在, 则 `influence_score = cci_norm * grn_weight_norm`。

所以如果要做真正的 inter CCI-only, 不能只把乘法改成 `cci_norm`, 还要取消 `pair_lookup` 对 inter edge 的强制过滤。否则虽然分数不乘 GRN, 但边是否存在仍然被 GRN 决定, 实验就不干净。

2. Legacy 网络消融设计

2.1 主实验只动 inter block，保留 intra GRN

建议主实验先只消融 inter block，因为用户当前问题主要指向“细胞-细胞之间 LR 配受体关系里 CCI 和 GRN 怎么融合”。这样可以隔离变量：

network method	intra block	inter block	目的
legacy_mixed_grn_cci	GRN	CCI × GRN	当前 baseline
legacy_inter_cci_only	GRN	CCI	检查 inter 里 GRN 是否真的有帮助
legacy_inter_additive_grn_cci	GRN	$(\alpha \text{ CCI} + \beta \text{ GRN}) / (\alpha + \beta)$	检查加法融合是否优于乘法融合

这里不建议一上来把 intra GRN 也删掉，因为那会把两个变量混在一起：一方面 inter 变了，另一方面 intra 也没了。主实验应该先回答“inter 的融合方式是否合理”。

2.2 可选严格对照：完全 CCI-only

如果导师明确要求“整个 legacy 网络都不能用 GRN”，再加一个可选 network method：

network method	intra block	inter block	目的
legacy_cci_only_no_intra	空 / 全零	CCI	完全去除 GRN 的严格 CCI-only 对照

这个方法可以作为附加实验，不建议作为第一轮主实验。

2.3 三种 inter influence 公式

对一个 inter edge：源 unit 为 u_i ，目标 unit 为 u_j ，ligand 为 L ，receptor 为 R 。

令：

$$s_{CCI} = \text{normalized COMMOT score}(u_i, u_j, L, R)$$

$$s_{GRN} = \text{normalized GRN weight}(L, R)$$

三种模式：

2.3.1 Product baseline

$$I = s_{CCI} \cdot s_{GRN}$$

保持当前逻辑。默认仍要求 (L, R) 存在于 GRN。

2.3.2 CCI-only

$$I = s_{CCI}$$

这个模式下 inter edge 不再要求 (L, R) 存在于 GRN。只要 COMMOT LR edge 存在，并且 ligand/receptor 表达条件通过，就保留。

2.3.3 Additive fusion

$$I = \frac{\alpha s_{CCI} + \beta s_{GRN}}{\alpha + \beta}$$

默认 $\alpha=1$, $\beta=1$ 。为了和 product baseline 公平比较, 第一版建议 additive 仍然要求 (L,R) 存在于 GRN。后面可以加 `inter_grn_pair_policy="zero_if_missing"`, 允许 GRN 缺失时把 $s_{GRN} = 0$ 。

3. 代码修改计划: Legacy 网络部分

3.1 修改 `mignet_ce/graph/builder.py`

3.1.1 新增参数 给 `build_layer_graph(...)` 增加参数:

```
inter_influence_mode: str = "product"
inter_additive_cci_weight: float = 1.0
inter_additive_grn_weight: float = 1.0
inter_grn_pair_policy: str = "require_pair"
include_intra_grn: bool = True
```

建议允许值:

```
inter_influence_mode in {"product", "cci_only", "additive"}
inter_grn_pair_policy in {"require_pair", "zero_if_missing"}
```

3.1.2 修改 `_build_commot_inter_edges(...)` 新增同样的参数, 并把当前硬编码的:

```
if (ligand_gene, receptor_gene) not in pair_lookup:
    continue
raw_w, norm_w = pair_lookup[(ligand_gene, receptor_gene)]
influence_score = cci_norm * norm_w
```

改成一个独立函数, 例如:

```
def _resolve_inter_influence(
    cci_norm: float,
    pair_key: tuple[str, str],
    pair_lookup: dict[tuple[str, str], tuple[float, float]],
    mode: str,
    additive_cci_weight: float,
    additive_grn_weight: float,
    grn_pair_policy: str,
) -> tuple[float | None, float | None, float | None]:
    ...
```

返回:

```
raw_w, norm_w, influence_score
```

不同模式的行为:

mode	GRN pair 不存在时	influence_score
product	跳过	$cci_norm * norm_w$
cci_only	不跳过	cci_norm
additive + require_pair	跳过	$(\alpha * cci_norm + \beta * norm_w) / (\alpha + \beta)$
additive + zero_if_missing	不跳过, $norm_w=0$	$(\alpha * cci_norm + \beta * 0) / (\alpha + \beta)$

3.1.3 控制 intra GRN 是否保留 当前:

```
intra = _build_intra_edges(...)
```

改成:

```
if include_intra_grn:
    intra = _build_intra_edges(...)
else:
    intra = pd.DataFrame(columns=EDGE_COLUMNS)
```

第一轮主实验的三个 network method 都让 `include_intra_grn=True`。

3.2 修改 mignet_ce/networks/legacy_mixed_grn_cci.py

不要复制三份完整 builder。建议把现有 class 改成可配置 builder:

```
class LegacyMixedGRNCCIBuilder:
    network_method = "legacy_mixed_grn_cci"
    inter_influence_mode = "product"
    inter_additive_cci_weight = 1.0
    inter_additive_grn_weight = 1.0
    inter_grn_pair_policy = "require_pair"
    include_intra_grn = True
```

在调用 build_layer_graph(...) 时传入这些参数。

然后新增两个轻量 subclass:

```
class LegacyInterCCIOnlyBuilder(LegacyMixedGRNCCIBuilder):
    network_method = "legacy_inter_cci_only"
    inter_influence_mode = "cci_only"
    inter_grn_pair_policy = "zero_if_missing"
    include_intra_grn = True

class LegacyInterAdditiveGRNCCIBuilder(LegacyMixedGRNCCIBuilder):
    network_method = "legacy_inter_additive_grn_cci"
    inter_influence_mode = "additive"
    inter_grn_pair_policy = "require_pair"
    include_intra_grn = True
```

可选再加:

```
class LegacyCCIOnlyNoIntraBuilder(LegacyMixedGRNCCIBuilder):
    network_method = "legacy_cci_only_no_intra"
    inter_influence_mode = "cci_only"
    inter_grn_pair_policy = "zero_if_missing"
    include_intra_grn = False
```

3.3 修改 network registry 和 config

3.3.1 mignet_ce/networks/registry.py 新增 import:

```
from mignet_ce.networks.legacy_mixed_grn_cci import (
    LegacyMixedGRNCCIBuilder,
    LegacyInterCCIOnlyBuilder,
    LegacyInterAdditiveGRNCCIBuilder,
)
```

注册:

```
NETWORK_BUILDERS = {
    "legacy_mixed_grn_cci": LegacyMixedGRNCCIBuilder,
    "legacy_inter_cci_only": LegacyInterCCIOnlyBuilder,
    "legacy_inter_additive_grn_cci": LegacyInterAdditiveGRNCCIBuilder,
    ...
}
```

3.3.2 mignet_ce/config.py 把 NETWORK_METHODS 改成:

```
NETWORK_METHODS = {
    "legacy_mixed_grn_cci",
    "legacy_inter_cci_only",
    "legacy_inter_additive_grn_cci",
    "cross_cell_multilayer",
    "expression_only",
}
```

可选再加入:

```
"legacy_cci_only_no_intra"
```

3.4 metadata 和导出

在 NetworkContext.metadata 里增加:

```
"legacy_inter_influence_mode": self.inter_influence_mode,
"legacy_inter_grn_pair_policy": self.inter_grn_pair_policy,
"legacy_include_intra_grn": self.include_intra_grn,
"legacy_additive_cci_weight": self.inter_additive_cci_weight,
"legacy_additive_grn_weight": self.inter_additive_grn_weight,
```

在 graph_summaries 中补充:

```

"inter_influence_mode": self.inter_influence_mode,
"include_intra_grn": self.include_intra_grn,
"inter_edges": len(graph.inter_edges),
"intra_edges": len(graph.intra_edges),

```

这样后面查结果时能知道每套实验到底用了哪种融合模式。

4. PIJ 新方法设计

4.1 当前已有方法

当前已经有：

PIJ method	成本项
expr_pseudotime_sr_ot	expression + pseudotime + SR
expr_pseudotime_sr_spatial_ot	expression + pseudotime + SR + spatial
expr_pseudotime_sr_energy_ot	expression + pseudotime + SR + graph energy

现在缺少：

新方法	成本项
expr_pseudotime_sr_energy_spatial_ot	expression + pseudotime + SR + graph energy + spatial

4.2 新 PIJ 公式

对源时间点 t 的 unit i 和目标时间点 $t + 1$ 的 unit j ，定义：

$$C_{expr}(i, j) = d(z_i^t, z_j^{t+1})$$

$$C_{pt}(i, j) = |\tau_i^t - \tau_j^{t+1}|$$

$$C_{sr}(i, j) = |sr_i^t - sr_j^{t+1}|$$

$$C_{energy}(i, j) = \left| \|z_i^t\|_2 - \|z_j^{t+1}\|_2 \right|$$

$$C_{spatial}(i, j) = \|x_i^t - x_j^{t+1}\|_2$$

总 cost：

$$C(i, j) = \frac{w_e \tilde{C}_{expr} + w_p \tilde{C}_{pt} + w_s \tilde{C}_{sr} + w_g \tilde{C}_{energy} + w_x \tilde{C}_{spatial}}{w_e + w_p + w_s + w_g + w_x}$$

其中每个 $\tilde{C}$ 都是归一化后的 cost。当前 `combine_costs(...)` 已经会做归一化和加权平均，不需要重复写归一化逻辑。

默认权重沿用现有 config：

component	config 字段	默认建议
expression	pij_expr_weight	1.0
pseudotime	pij_pseudotime_weight	0.5
SR	pij_sr_weight	0.5
graph energy	pij_graph_energy_weight	0.2
spatial	pij_spatial_weight	0.2

第一轮建议跑一个小权重 sweep:

```
pij_spatial_weight = 0.05, 0.10, 0.20, 0.50
```

这样可以判断 spatial 是真的提升，还是权重过大导致空间邻近性压过表达/发育信息。

4.3 新增文件

新增:

mignet_ce/pij/expr_pseudotime_sr_energy_spatial_ot.py

结构直接参考 expr_pseudotime_sr_energy_ot.py 和 expr_pseudotime_sr_spatial_ot.py。

核心 components:

```
components = {
    "expression": build_expression_cost(source_features, target_features, cfg),
    "pseudotime": build_pseudotime_cost(source_table, target_table, self.name),
    component_name: sr_cost,
    "graph_energy": build_graph_energy_cost(source_features, target_features),
    "spatial": build_spatial_cost(source_coords, target_coords, self.name),
}
weights = {
    "expression": cfg.pij_expr_weight,
    "pseudotime": cfg.pij_pseudotime_weight,
    component_name: cfg.pij_sr_weight,
    "graph_energy": cfg.pij_graph_energy_weight,
    "spatial": cfg.pij_spatial_weight,
}
```

metadata:

```
metadata = developmental_metadata(source_table, target_table, ["pseudotime", sr_column], cfg)
```

如果 source_coords 或 target_coords 缺失, build_spatial_cost(...) 会报错, 这是合理的。

4.4 注册新 PIJ method

mignet_ce/pij/registry.py 新增 import:

```
from mignet_ce.pij.expr_pseudotime_sr_energy_spatial_ot import ExprPseudotimeSREnergySpatialOTPijMethod
```

注册:

```
"expr_pseudotime_sr_energy_spatial_ot": ExprPseudotimeSREnergySpatialOTPijMethod,
```

mignet_ce/config.py 在 PIJ_METHODS 加入:

```
"expr_pseudotime_sr_energy_spatial_ot",
```

在 DEVELOPMENT_PIJ_METHODS 加入同名方法, 因为它需要 development_feature_root。

在 PIJ_METHOD_PRESETS 加入一个新 preset:

```
"ot_ablation_v4": (
    "expr_pseudotime_sr_energy_ot",
    "expr_pseudotime_sr_energy_spatial_ot",
),
```

同时把它加入 all。

5. 实验矩阵建议

5.1 第一轮：只验证新网络消融是否能跑通

先固定一个 PIJ method，例如：

```
expr_pseudotime_sr_energy_ot
```

跑三个 network method：

```
legacy_mixed_grn_cci  
legacy_inter_cci_only  
legacy_inter_additive_grn_cci
```

目的：确认 network variant 的 EI/DI 是否能稳定产出。

5.2 第二轮：加入新 PIJ method

network method 保持三个：

```
legacy_mixed_grn_cci  
legacy_inter_cci_only  
legacy_inter_additive_grn_cci
```

PIJ method 使用：

```
expr_pseudotime_sr_energy_ot  
expr_pseudotime_sr_energy_spatial_ot
```

这样可以回答两个问题：

1. inter 融合方式：product、CCI-only、additive 谁更好？
2. PIJ 中加入 spatial 后，是否比 expression + pseudotime + SR + energy 更好？

5.3 第三轮：解释性对照

为了避免把所有提升都归因于 spatial，可以额外加入：

```
expr_pseudotime_sr_spatial_ot  
spatial_ot
```

如果 spatial_ot 单独也很高，需要警惕结果是否主要来自空间邻近性，而不是发育/表达/网络信息。

6. 推荐命令

6.1 Smoke test

先只跑 heart、一个 pair、两个时间点：

```
python -B scripts/run_mignet_vertical_ablation.py \  
--data-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory \  
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_legacy_fusion_smoke" \  
--network-methods legacy_mixed_grn_cci legacy_inter_cci_only legacy_inter_additive_grn_cci \  
--pij-methods expr_pseudotime_sr_energy_ot expr_pseudotime_sr_energy_spatial_ot \  
--organs heart \  
--time-points 11.5 12.5 \  
--level-pairs spot:seurat_less_than5 \  
--development-feature-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/developmental_features \  
--no-export-features \  
--export-pair-artifacts \  
--export-pij \  
--pij-archive-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/pij
```

6.2 Seurat 主实验

注意：按照之前要求，先不要包含 seurat_less_than5:seurat_k40。

```
python -B scripts/run_mignet_vertical_ablation.py \  
--data-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory \  
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_legacy_fusion_seurat" \  
--network-methods legacy_mixed_grn_cci legacy_inter_cci_only legacy_inter_additive_grn_cci \  
--pij-methods expr_pseudotime_sr_energy_ot expr_pseudotime_sr_energy_spatial_ot \  
--organs heart brain lung \  
--seurat-less-than5 seurat_k40
```

```
--time-points 11.5 12.5 13.5 14.5 \
--level-pairs \
  spot:seurat_less_than5 \
  seurat_less_than5:seurat_k150 \
  seurat_k150:seurat_k40 \
  spot:seurat_k150 \
  spot:seurat_k40 \
--development-feature-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/developmental_features \
--export-pij \
--pij-archive-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/pij
```

6.3 Louvain 主实验

如果要在 louvain 上做同样实验，用 louvain 对应层级。为了和 seurat 保持一致，第一轮也可以先不放 louvain_less_than5:louvain_k40，只保留 adjacent 和 spot-to-k 层级：

```
python -B scripts/run_mignet_vertical_ablation.py \
--data-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_legacy_fusion_louvain" \
--network-methods legacy_mixed_grn_cci legacy_inter_cci_only legacy_inter_additive_grn_cci \
--pij-methods expr_pseudotime_sr_energy_ot expr_pseudotime_sr_energy_spatial_ot \
--organs heart brain lung \
--time-points 11.5 12.5 13.5 14.5 \
--level-pairs \
  spot:louvain_less_than5 \
  louvain_less_than5:louvain_k150 \
  louvain_k150:louvain_k40 \
  spot:louvain_k150 \
  spot:louvain_k40 \
--development-feature-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/developmental_features \
--export-pij \
--pij-archive-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/pij
```

6.4 Spatial weight sweep

示例：

```
for w in 0.05 0.10 0.20 0.50; do
python -B scripts/run_mignet_vertical_ablation.py \
--data-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_legacy_fusion_spatial_w${w}" \
--network-methods legacy_mixed_grn_cci legacy_inter_cci_only legacy_inter_additive_grn_cci \
--pij-methods expr_pseudotime_sr_energy_spatial_ot \
--organs heart brain lung \
--time-points 11.5 12.5 13.5 14.5 \
--level-pairs spot:seurat_less_than5 seurat_less_than5:seurat_k150 seurat_k150:seurat_k40 spot:seurat_k150
  ↪ spot:seurat_k40 \
--development-feature-root /home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/developmental_features \
--pij-spatial-weight "$w"
done
```

7. 验收标准

7.1 代码层面

- python -m compileall mignet_ce scripts 无语法错误。
- scripts/run_mignet_vertical_ablation.py --help 能看到新增 network method 和 PIJ method。
- 新 network methods 能被 NETWORK_METHODS、NETWORK_BUILDERS 正确识别。
- 新 PIJ method 能被 PIJ_METHODS、PIJ_METHOD_REGISTRY 正确识别。

7.2 数据导出层面

开启 --export-pij 后，每个实验应输出：

```
pij/network=<network_method>/pij=<pij_method>/organ=<organ>/pair=<pair>/units.csv
pij/network=<network_method>/pij=<pij_method>/organ=<organ>/pair=<pair>/*_lower_P.npz
pij/network=<network_method>/pij=<pij_method>/organ=<organ>/pair=<pair>/*_upper_P.npz
pij/network=<network_method>/pij=<pij_method>/organ=<organ>/pair=<pair>/kernel_metadata.json
```

并且 .npz 是完整 PIJ，不是 top-k。

7.3 实验解释层面

最终比较至少输出：

对比	看什么
legacy_mixed_grn_cci vs legacy_inter_cci_only	inter block 中 GRN 是否有贡献
legacy_mixed_grn_cci vs legacy_inter_additive_grn_cci	乘法融合还是加法融合更好
expr_pseudotime_sr_energy_ot vs expr_pseudotime_sr_energy_spatial_ot	spatial 是否进一步提升 PIJ
不同 pij_spatial_weight	spatial 是否过强、是否造成空间主导

核心指标：

EI_gain mean / median / positive ratio
DI_mean / median
TE_mean / median
run_summary status

8. 需要特别注意的坑

1. **CCI-only** 不能继续用 GRN pair_lookup 过滤 edge，否则不是纯 CCI-only。
2. 主实验建议保留 intra GRN，只改 inter block，否则变量太多，解释会混乱。
3. **additive** 第一版建议仍要求 GRN pair 存在，这样和 baseline 的 edge set 更接近，只比较乘法和加法。
4. **expr_pseudotime_sr_energy_spatial_ot** 需要 **coordinates**。如果某个层级坐标缺失，应该直接报错，而不是静默退化。
5. **spatial** 权重必须 **sweep**。如果 w_spatial=0.5 或更大效果很好，不一定说明模型更好，也可能说明空间邻近性压过了发育/表达信号。
6. 导出 PIJ 时确认 **full_matrix=True**，不要只看 top-k CSV。

9. 建议 Codex 执行顺序

1. 先改 mignet_ce/graph/builder.py，把 inter influence 逻辑抽成函数。
2. 再改 legacy_mixed_grn_cci.py，通过 subclass 注册三个 legacy variants。
3. 改 networks/registry.py 和 config.py，让 CLI 能识别新增 network methods。
4. 新增 expr_pseudotime_sr_energy_spatial_ot.py。
5. 改 pij/registry.py 和 config.py，注册新 PIJ method 和 preset。
6. 跑 compileall。
7. 跑 smoke test。
8. 检查 run_summary.csv、metrics.csv、kernel_metadata.json、units.csv 和完整 .npz。
9. smoke test 通过后，再跑 seurat/louvain 全量实验。